



SAFECHILD

REGULATE • CONNECT • PROTECT

Software & System Architecture Documentation

TekMen Revolution Development Team

Version 1.0 | May 2026

Our mission is simple:

A safer world for every child, connected by community and powered by technology.



Table of Contents

1. System Overview & Core Workflow.....	3
1.1 Global System Data Flow.....	3
2. Supabase Database Schema.....	4
2.1 Tables at a Glance.....	4
2.2 SQL Table Definitions.....	4
3. Hardware-to-Backend Interface (API Specification).....	6
3.1 JSON Payload Structure (Device to Server).....	6
3.2 Server Evaluation Logic & Status Codes.....	6
3.3 Edge Function Pseudocode.....	7
4. Phase 2 — Cross-Reference Matching Engine.....	8
4.1 Algorithmic Matching Weights.....	8
4.2 Matching Flow.....	8
5. Privacy-First Data Guardrails.....	9
6. Frontend Routing & State Architecture.....	10
6.1 Main Navigation Structure.....	10
6.2 Screen Specifications.....	10
6.3 Technical Stack.....	11
A. Appendix — Quick Reference.....	12
A.1 Device Status Codes.....	12
A.2 Full Technology Stack.....	12



1 System Overview & Core Workflow

SafeChild is a dual-phase, community-driven child safety ecosystem. It bridges physical hardware IoT tracking with a real-time, crowd-sourced mobile network. The core principle: a child's location is private by default and only surfaces when the device enters an alert state.

1.1 Global System Data Flow

The linear data lifecycle moves from low-level hardware packets up to encrypted application states:

#	Component	Action
1	Wearable Device (SIM808)	Transmits GSM/GPRS uplink with GPS coordinates and sensor readings
2	Cellular Tower Network	Routes signal to the public internet
3	Supabase Edge Function	Receives HTTPS POST at /v1/telemetry and evaluates device state
4	Supabase Realtime DB	Persists telemetry log and updates the device status record
5	Cross-Reference Engine	Async DB trigger fires on new incident_reports to run the matching algorithm
6	Firebase Cloud Messaging	Broadcasts push notification only on SOS or low battery state
7	React Native App	Receives push notification, updates AlertsScreen and HomeScreen
8	Parent / Community Network	Views alert details, location, and algorithmic match results

 **Privacy Note:** Location data is *ONLY* broadcast when a device enters an SOS or low battery state. No passive ambient location streaming occurs under any circumstance.

2 Supabase Database Schema

The relational structure maps entity dependencies. The wearable_logs tracking table operates independently from the child's public-facing metadata to enforce privacy-first protocols. Row-Level Security is applied to all sensitive tables.

2.1 Tables at a Glance

Table Name	Phase	Purpose
profiles	1 & 2	Parent and community user accounts and settings
children	1 & 2	Child metadata, linked to parent profiles
wearables	1	Hardware device registry with geofence config
wearable_logs	1	Privacy-gated telemetry archive (RLS protected)
incident_reports	2	Community missing/found child reports

2.2 SQL Table Definitions

profiles

```
CREATE TABLE profiles (  
  id          UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,  
  full_name   TEXT NOT NULL,  
  phone_number TEXT NOT NULL UNIQUE,  
  community_mode TEXT CHECK (community_mode IN ('active', 'passive')) DEFAULT  
'passive',  
  reputation_points INT DEFAULT 0,  
  created_at   TIMESTAMP WITH TIME ZONE DEFAULT NOW() NOT NULL  
);
```

children

```
CREATE TABLE children (  
  id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  parent_id   UUID REFERENCES profiles(id) ON DELETE CASCADE,  
  first_name  TEXT NOT NULL,  
  last_name   TEXT NOT NULL,  
  age         INT NOT NULL,  
  gender      TEXT NOT NULL,  
  photo_url   TEXT,  
  is_missing  BOOLEAN DEFAULT FALSE,  
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW() NOT NULL  
);
```

wearables

```
CREATE TABLE wearables (  
  id                UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  child_id          UUID REFERENCES children(id) ON DELETE SET NULL,  
  imei_number       VARCHAR(15) UNIQUE NOT NULL,  
  sim_phone_number  TEXT UNIQUE NOT NULL,  
  safe_zone_center_lat NUMERIC(10, 8),  
  safe_zone_center_lng NUMERIC(11, 8),  
  safe_zone_radius_meters INT DEFAULT 200,  
  device_status     TEXT CHECK (device_status IN (  
    'normal', 'sos_manual', 'sos_geofence', 'sos_tamper', 'low_battery'  
  )) DEFAULT 'normal',  
  last_updated      TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

wearable_logs (RLS protected)

```
CREATE TABLE wearable_logs (  
  id                BIGSERIAL PRIMARY KEY,  
  wearable_id       UUID REFERENCES wearables(id) ON DELETE CASCADE,  
  latitude          NUMERIC(10, 8) NOT NULL,  
  longitude         NUMERIC(11, 8) NOT NULL,  
  battery_level     INT NOT NULL,  
  is_worn           BOOLEAN DEFAULT TRUE,  
  logged_at         TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

incident_reports (Phase 2)

```
CREATE TABLE incident_reports (  
  id                UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  reporter_id       UUID REFERENCES profiles(id) ON DELETE SET NULL,  
  child_id          UUID REFERENCES children(id) ON DELETE SET NULL,  
  report_type       TEXT CHECK (report_type IN ('missing', 'found')) NOT NULL,  
  distinct_features TEXT,  
  last_seen_location_text TEXT,  
  lat               NUMERIC(10, 8),  
  lng               NUMERIC(11, 8),  
  is_resolved       BOOLEAN DEFAULT FALSE,  
  created_at        TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

3 Hardware-to-Backend Interface (API Specification)

The SIM808 module communicates with the backend via a secure HTTPS POST payload hitting a Supabase Edge Function endpoint at /v1/telemetry. The IMEI number is the device identifier, validated server-side against the wearables registry.

3.1 JSON Payload Structure (Device to Server)

```
{
  "imei": "358081023456789",
  "timestamp": "2026-05-21T11:50:00Z",
  "gps": {
    "lat": 4.154200,
    "lng": 9.241500,
    "altitude": 540.2
  },
  "sensors": {
    "battery_percentage": 14,
    "optical_skin_contact": false
  },
  "trigger": "tamper_alert"
}
```

3.2 Server Evaluation Logic & Status Codes

Status	Trigger Condition	Consequence
sos_manual	trigger === 'manual_button'	Immediate FCM broadcast to parent and responders
sos_tamper	optical_skin_contact === false	FCM broadcast — wrist removal tamper alert
low_battery	battery_percentage < 15	FCM broadcast — low battery warning
sos_geofence	Haversine distance > safe_zone_radius_meters	FCM broadcast — geofence breach
normal	No conditions matched	Silent write to wearable_logs, no broadcast

3.3 Edge Function Pseudocode

```
exports.handler = async (req, res) => {
  const { imei, gps, sensors, trigger } = req.body;

  // 1. Fetch device record from wearables registry
  const { data: device } = await supabase
    .from('wearables').select('*').eq('imei_number', imei).single();
```

```
let status = 'normal';

// 2. Evaluate conditional danger rules (priority order)
if (trigger === 'manual_button')      status = 'sos_manual';
else if (!sensors.optical_skin_contact) status = 'sos_tamper';
else if (sensors.battery_percentage < 15) status = 'low_battery';
else {
  const dist = haversine(gps.lat, gps.lng,
    device.safe_zone_center_lat, device.safe_zone_center_lng);
  if (dist > device.safe_zone_radius_meters) status = 'sos_geofence';
}

// 3. Always write to telemetry log
await supabase.from('wearable_logs').insert([
  {wearable_id: device.id, latitude: gps.lat, longitude: gps.lng,
    battery_level: sensors.battery_percentage, is_worn: sensors.optical_skin_contact}
]);

// 4. Update device state record
await supabase.from('wearables')
  .update({ device_status: status, last_updated: 'now()' })
  .eq('id', device.id);

// 5. PRIVACY GUARDRAIL — broadcast only on alert states
if (status.startsWith('sos') || status === 'low_battery')
  await triggerFCMBroadcast(device.id, status, gps.lat, gps.lng);

return res.status(200).json({ success: true, status });
};
```

✓ **Security Note:** All data is encrypted in transit and at rest. IMEI-based device auth is validated against the registry on every request.

4 Phase 2 — Cross-Reference Matching Engine

When a new record is added to incident_reports, an asynchronous database trigger runs a similarity cross-reference across opposing report types. A 'missing' report is scored against all unresolved 'found' reports, and vice versa.

4.1 Algorithmic Matching Weights

Parameter	Metric	Mechanism	Weight
Gender	Exact match	String equality check	30%
Age	Delta deviation	±2 year tolerance window	20%
Geographic proximity	Spatial distance	Haversine formula — max 15 km radius	40%
Distinct features	Lexical match	Vector embeddings + cosine similarity scoring	10%

 **Alert Threshold:** *If the cumulative score exceeds 75%, the system generates an alert entry and immediately notifies the parent and local emergency responders via Firebase Cloud Messaging.*

4.2 Matching Flow

- A new incident_reports record is inserted with report_type 'missing' or 'found'.
- The trigger queries all unresolved, opposite-type reports within a 15 km radius.
- Each candidate is scored across all four parameters using the weights above.
- Candidates scoring above 75% generate a new alert record with match breakdown.
- FCM broadcasts to the parent and all registered local responders.

5 Privacy-First Data Guardrails

To protect minors from malicious tracking or data interception, the technical design explicitly bans passive ambient location streaming. These constraints are non-negotiable and enforced at the database level, not the application layer.

Guardrail	Implementation
At-rest encryption	All raw coordinate pairs in <code>wearable_logs</code> are isolated behind PostgreSQL Row-Level Security (RLS) layers. No direct client access to the raw table.
Read access control	The React Native client runtime has NO fetch access to the historic tracking log unless <code>wearables.device_status</code> shifts out of the 'normal' state. Enforced via RLS, not application code.
Session auto-teardown	The moment an alert transitions to <code>is_resolved = true</code> , real-time location streaming terminates instantly across all client apps.
No passive streaming	Location coordinates are never included in FCM payloads during normal operation. Broadcasts are gated exclusively to SOS and <code>low_battery</code> states.

6 Frontend Routing & State Architecture

The user interface maps clean application components to functional routes inside the root navigation layer. All alert data is kept in sync via Supabase Realtime WebSocket subscriptions — no polling.

6.1 Main Navigation Structure

```
AppRoot
├── AuthStack
│   ├── LoginScreen
│   └── RegistrationScreen
└── BottomTabNavigator
    ├── HomeScreen      — Urgent local incidents, 'I Can Help' toggle
    ├── ReportsScreen   — Report Missing / Report Found grid
    ├── AlertsScreen    — SOS alert listings + algorithmic match results
    └── ProfileScreen   — Reputation ledger, app preferences
```

6.2 Screen Specifications

Screen	Functionality
HomeScreen	Displays urgent incidents filtered by user proximity. Provides the 'I Can Help' toggle that switches the user into active community mode.
ReportsScreen	Two-tile grid view: 'Report Missing Child' and 'Report Found Child'. Both write to incident_reports and trigger the Phase 2 matching engine.
AlertsScreen	Lists all active SOS alerts and algorithm correlation matches. High-priority sorted. Resolves when a parent confirms the outcome.
ProfileScreen	Shows reputation reward ledger and earned badges. Houses notification and location permission preferences.

6.3 Technical Stack

Concern	Technology	Notes
State management	React Context / Redux Toolkit	User location permissions and notification toggles
Map rendering	react-native-maps	Geofence visualization and incident location pins
Background location	Native Android & iOS hooks	Fires geofence breach events while app is backgrounded
Push notifications	Firebase Cloud	SOS alerts, match notifications, low-battery



	Messaging	warnings
Auth & sessions	Supabase Auth (JWT)	Session tokens used in RLS policy enforcement
Live updates	Supabase Realtime (WebSocket)	Device status and alert records sync without polling

A Appendix — Quick Reference

A.1 Device Status Codes

Code	Meaning	FCM Broadcast
normal	Device operating normally within safe zone	No
sos_manual	Child pressed the physical SOS button	Yes — immediate
sos_geofence	Child's device left the designated safe zone	Yes — immediate
sos_tamper	Optical sensor lost skin contact (removal)	Yes — immediate
low_battery	Battery dropped below 15%	Yes — warning

A.2 Full Technology Stack

Layer	Technology	Role
Hardware	SIM808 Module	GSM/GPRS modem with integrated GPS receiver
Backend DB	Supabase (PostgreSQL)	Database, auth, edge functions, realtime
Messaging	Firebase Cloud Messaging	Push notifications and SOS broadcasts
Mobile App	React Native	Cross-platform iOS and Android client
Maps & Geo	react-native-maps	Geofencing UI and incident location display
Match AI	Vector Embeddings + Cosine Similarity	Phase 2 incident cross-reference engine